CS588 MP2: Center-Based 3D Detection From LiDAR

Due date: 11:59pm, Monday April 20th, 2026 on Canvas

Goal

In this MP you will implement a small but complete CenterPoint-style detector for single-sweep Velodyne LiDAR on KITTI raw sequences with tracklets [7]. You will learn and implement every steps of the pipelines, from data processing, model architecture, hyperparamters tuning, and evaluation as a true ML engineer working in Autonomous Driving.

> Please do not distribute the data to outside the classroom in compliant with KITTI's policy. Please check the HW announcement for the link to download the data.

Representation Contract

Coordinate System and Grid Sizes

We use the LiDAR frame throughout the assignment: x points forward, y points left, and z points up. Each box is represented as

$$\mathbf{b} = [x,\ y,\ z,\ l,\ w,\ h,\ \theta],$$

where (x, y, z) is the box center, l is length along the local forward axis, w is width, h is height, and θ is yaw in radians.

The BEV region of interest is fixed to

$$x \in [0.0, 70.4], \qquad y \in [-40.0, 40.0], \qquad z \in [-3.0, 1.0].$$

Let the BEV raster resolution be Δ meters per pixel and the output stride be s. Then

$$W = \frac{x_{\max} - x_{\min}}{\Delta}, \qquad H = \frac{y_{\max} - y_{\min}}{\Delta}, \qquad W_{\text{out}} = \frac{W}{s}, \qquad H_{\text{out}} = \frac{H}{s}.$$

For the homework configuration, $\Delta = 0.2$ m and $s = 4$. Therefore the input grid is $H \times W = 400 \times 352$, the output grid is $H_{\text{out}} \times W_{\text{out}} = 100 \times 88$, and each output cell covers 0.8 m $\times$ 0.8 m.

> **Sanity check:** If your tensor sizes are not $(4, 400, 352)$ for the model input and $(100, 88)$ for each output head, some coordinate transform is wrong.

To map a point (x, y) into the BEV image, use

$$c = \left\lfloor \frac{x - x_{\min}}{\Delta} \right\rfloor, \qquad r = (H - 1) - \left\lfloor \frac{y - y_{\min}}{\Delta} \right\rfloor.$$

To map an object center (x, y) into the stride-s output grid, use

$$u = \frac{x - x_{\min}}{s\Delta}, \qquad v = (H_{\text{out}} - 1) - \frac{y - y_{\min}}{s\Delta}.$$

The integer cell is $(\lfloor u \rfloor, \lfloor v \rfloor)$, and the sub-cell residual is $(u - \lfloor u \rfloor,\ v - \lfloor v \rfloor)$.

Input Tensor: Four BEV Channels

Let $P = \{(x_i, y_i, z_i, r_i)\}_{i=1}^{N}$ denote one LiDAR sweep after filtering to the valid ROI and z-range. The assignment uses exactly four input channels, in this order:

Channel	Definition
`max_height`	Per-cell maximum z, normalized over $[z_{\min}, z_{\max}]$ to $[0, 1]$. Empty cells write 0.
`mean_height`	Per-cell mean z, normalized over $[z_{\min}, z_{\max}]$ to $[0, 1]$. Empty cells write 0.
`intensity`	Per-cell mean LiDAR reflectance, clipped to $[0, 1]$. Empty cells write 0.
`density`	$\log(1 + n)$ point count per cell, divided by $\log(1 + 64) = \log(65)$, clipped to $[0, 1]$. In code: `np.log1p(counts) / np.log1p(64.0)`.

Inside the model and dataloader, the BEV tensor uses PyTorch convention $(C_{\text{in}}, H, W) = (4, 400, 352)$. Inside the processed `.npz` artifact, the same tensor is stored as $(W, H, C_{\text{in}}) = (352, 400, 4)$. Do not mistake storage layout for model layout.

Output Tensor: $K + 9$ Dense Channels

Let K be the number of semantic classes ($K = 1$ in this homework, merged into `Car`). For one object m with box $[x_m, y_m, z_m, l_m, w_m, h_m, \theta_m]$, compute the continuous output-grid center (u_m, v_m), the integer cell $(\bar{u}_m, \bar{v}_m)$, and the sub-cell offset $\delta u_m = u_m - \bar{u}_m$, $\delta v_m = v_m - \bar{v}_m$. The dense target tensor is $T \in \mathbb{R}^{10\times100\times88}$ in PyTorch format and $(88, 100, 10)$ in the processed `.npz` file.

Channel group	Shape	Meaning
`heatmap_{class}`	$K \times 100 \times 88$	Per-class center heatmap with Gaussian bumps at object centers.
`offset_x`, `offset_y`	$2 \times 100 \times 88$	Sub-cell residuals $(\delta u_m, \delta v_m)$, written only at the object-center cell.
`center_z`	$1 \times 100 \times 88$	Metric z-coordinate of the object center.
`log_length`, `log_width`, `log_height`	$3 \times 100 \times 88$	Box dimensions stored as $(\log l_m, \log w_m, \log h_m)$ (use_log_dims=True).
`sin_yaw`, `cos_yaw`	$2 \times 100 \times 88$	Continuous yaw representation $(\sin\theta_m, \cos\theta_m)$.
`reg_mask`	$1 \times 100 \times 88$	Binary mask marking which cells have a valid object center (used for regression losses).

Only the heatmap is spatially dense. The regression channels are sparse and supervised only at the discrete center cell for each object.

Loss Function

The training objective is a weighted sum of one CenterNet-style focal loss for the heatmap and masked L_1 losses for the regression heads [6, 5]:

$$\mathcal{L} = \lambda_{\text{hm}}\mathcal{L}_{\text{focal}}(\sigma(\hat{H}), H) + \lambda_{\text{off}}\mathcal{L}_1(\hat{O}, O \mid M) + \lambda_z\mathcal{L}_1(\hat{Z}, Z \mid M) + \lambda_{\text{dim}}\mathcal{L}_1(\hat{D}, D \mid M) + \lambda_{\text{yaw}}\mathcal{L}_1(\hat{R}, R \mid M). \tag{1}$$

where σ is sigmoid. The focal term follows:

$$\mathcal{L}_{\text{focal}}(p, H) = -\frac{1}{\max(N_{\text{pos}},\, 1)} \sum \Big[\mathbf{1}_{H=1}(1-p)^2 \log p + \mathbf{1}_{H<1}(1-H)^4 p^2 \log(1-p)\Big],$$

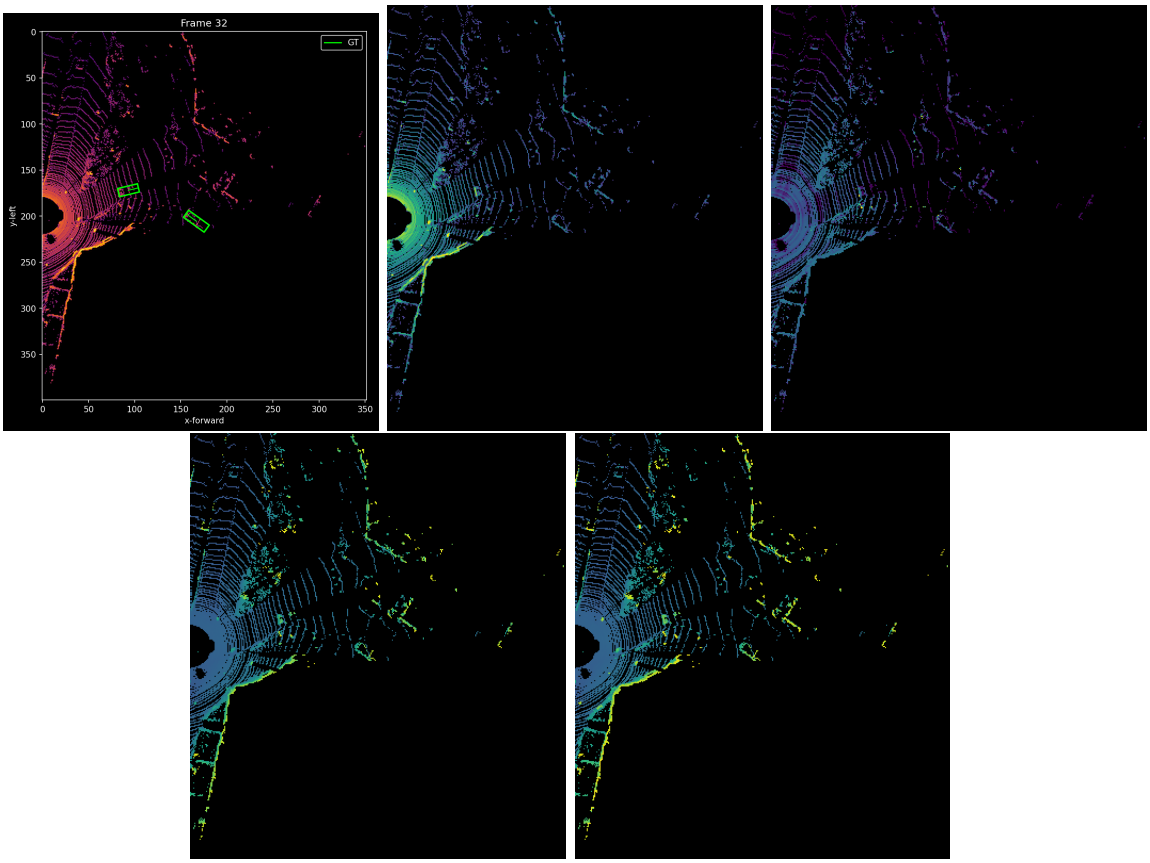

Figure 1: Example of how correct rasterization looks like per channel. From left to right, up to down: `bbox_overlaid`, `density`, `intensity`, `mean_height`, `max_height`

and the masked regression loss is

$$\mathcal{L}_1(\hat{A}, A \mid M) = \frac{1}{N_{\text{mask}}} \left\| M \odot (\hat{A} - A) \right\|_1 .$$

All loss weights are 1.0 in the reference configuration. The corresponding code lives in `lidar_det/models/simple_centerpoint.py` inside `compute_losses`.

Task 0: BEV Rasterization + Visualization [2 pts]

In this task, you will implement a rasterization function similar in MP1 to convert the point cloud into a 4-channel BEV tensor as described above.

What you need to implement:

- `rasterize_points_to_bev` in `lidar_det/data/bev.py` — rasterizes a LiDAR sweep into the 4-channel BEV tensor.

Implementation hints: Use `metric_to_grid` (already provided) to map (x, y) to (row, col) and filter by z-range. Compute a flat cell index flat_idx $= \text{row} \times W + \text{col}$ and use `np.bincount` for counts and weighted sums and `np.maximum.at` for per-cell max-height. Do **not** use nested Python loops over cells. See the in-code comments for detailed channel definitions and normalization.

To verify, run:

```
python scripts/00_read_and_viz_kitti_raw.py
```

The script loads raw KITTI data, calls your `rasterize_points_to_bev`, and prints diagnostics. Correct output looks like:

```
--- BEV diagnostics (frame 0000) ---
  shape: (4, 400, 352) (expected (4, 400, 352))
  occupancy: ~10000/140800 cells (~7%)
  max_height : range=[0.2, 1.0] mean=~0.4
  mean_height : range=[0.2, 1.0] mean=~0.4
  intensity : range=[0.0, 1.0] mean=~0.2
  density : range=[0.2, 1.0] mean=~0.4
```

What to check:

- **BEV shape** must be $(4, 400, 352)$. If not, your coordinate mapping is wrong.
- **Occupancy** should be roughly 5–10%. If 0%, your rasterization produced an empty tensor. If 100%, you are not zeroing empty cells after normalization.
- **All channel ranges** should be in $[0, 1]$ with no channel stuck at all zeros.

Check the saved images under `outputs/plots/01_kitti_raw/`. A correct visualization shows boxes lying on occupied LiDAR structure rather than floating far away, mirrored left-right, or rotated by 90°. An example of what it looks like is Fig. 1.

Task 1: Target Encoding [4 pts]

Now that your BEV rasterization works, your goal is to turn metric boxes into dense center-based supervision targets. The purpose of this is to preprocess the raw data into a supervision tensor as described above.

What you need to implement:

- `encode_targets` in `lidar_det/data/targets.py` — encodes GT boxes as CenterPoint-style sparse supervision targets.
- `decode_targets` in `lidar_det/data/targets.py` — decodes encoded targets back to metric boxes (used for debugging consistency).

To verify, run:

```
python scripts/01_debug_dataloader_and_targets.py
```

This script loads raw KITTI data, calls your BEV rasterization (Task 0) and `encode_targets`, `decode_targets`, and prints diagnostics. Correct output looks like:

```
--- Encode diagnostics (frame 0000) ---
  heatmap shape: (1, 100, 88) (expected (1, 100, 88))
  heatmap max: 1.0000 (expected 1.0 at object centers)
  encoded objects: 1 (gt boxes: 1)

frame=0000 gt=1 decoded=1 mean_center_err=0.0000m
```

```
...
PASS: encode/decode round-trip is consistent (error < 0.5m)
```

What to check:

- **Heatmap max** should be 1.0. If 0.0, your `draw_gaussian` is not being called or the center is out of bounds.
- **Encoded objects** should match **gt boxes**. If 0, your `encode_targets` is not filling any slots.
- **mean_center_err** should be ≈ 0 for all frames. If large or `inf`, the encode/decode round-trip is broken.

Also check the saved images under `outputs/plots/02_targets/`: green boxes are ground truth, red boxes are decoded from your encoded targets. They should overlap perfectly, so you might only see one color.

Implementation Steps:

1. In `encode_targets`: for each GT box, call `metric_to_output_grid` to locate its center on the stride-4 grid, then call `draw_gaussian` to place a Gaussian bump on the heatmap. Write the regression values $(\delta u, \delta v, z, \log l, \log w, \log h, \sin\theta, \cos\theta)$ only at the integer center cell and mark that cell in `mask`.
2. In `decode_targets`: invert the encoding — gather the regression values at each masked center index, recover metric (x, y) with `output_grid_to_metric`, exponentiate log-dimensions if `use_log_dims=True`, and recover yaw as $\theta = \text{atan2}(\sin\theta, \cos\theta)$.

Task 2: Model + Loss + Training Loop [5 pts]

Now that the dataloading part is done, it's time to move on to model architecture. Implement the CenterPoint [7] architecture following Fig 2, the loss functions, and the core training step.

Disclaimer: For the sake of saving storage/time and reproducibility, we provide a processed data for you, which you can used from this task.

What you need to implement: The following functions in `lidar_det/models/simple_centerpoint.py`:

- `SimpleCenterPoint.__init__` and `forward` — construct the residual backbone layers and wire them in the correct order. The `ResidualBlock` class is already provided.
- `_focal_loss_centerpoint` — the CenterNet focal loss.
- `_reg_l1_loss` — the masked L_1 regression loss.
- `compute_losses` — assembles the weighted multi-task loss.

- `train_step` — one gradient-update step (forward, loss, backward, optimizer step). The provided training script `02_train_processed_detector.py` calls your `train_step` inside its epoch loop; everything else (data loading, checkpointing, evaluation, logging) is handled for you. For a general introduction to PyTorch training loops, see https://docs.pytorch.org/tutorials/beginner/introyt/trainingyt.html.

To verify, train on the tiny `minival` split and confirm the model can overfit it. **IMPORTANT**: make sure to use the exact command as below to overfit on the minival split to avoid confusion.

```
python scripts/02_train_processed_detector.py \
  --train-split minival \
  --val-split minival \
  --test-split minival \
  --epochs 50 --batch-size 2 \
  --run-name task2_minival_debug
```

A successful overfit run should show output similar to:

```
run_dir=outputs/processed_train/task2_minival_debug
best_val_mAP=0.989011
test_mAP=0.989011
```

The exact mAP values and wall-clock time will vary depending on your hardware, but you should see non-zero mAP and decreasing training loss. If the model cannot overfit `minival`, your first suspects should be tensor layout, the center-index gather logic, and the masking in the regression losses. As long as the `test_mAP` reaches past 0.9, you are good.

The backbone and residual block architectures are shown in Figures 2 and 3. All convolutions in the backbone use padding=1 and no bias. The stem maps $4 \to 64$ channels at stride 1, followed by residual blocks producing a stride-4 feature map of shape $128 \times 100 \times 88$. Each prediction head (already provided) is a 3×3 conv $\to$ ReLU $\to$ 1×1 conv. The heatmap head's final bias is initialised to -2.19 for focal-loss stability.

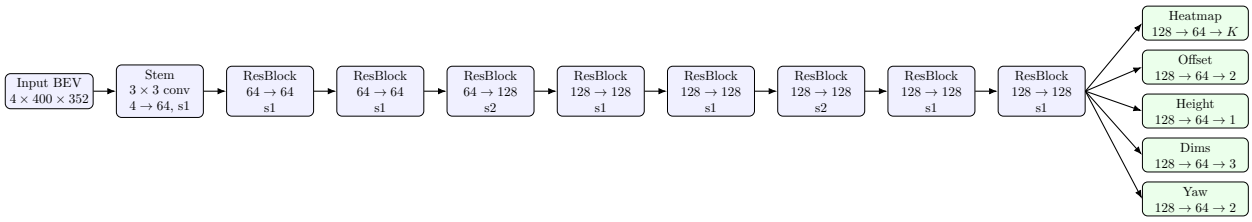

Figure 2: Full backbone architecture. "s1"/"s2" denotes stride. The two stride-2 blocks provide the overall $4\times$ spatial downsampling. Each prediction head is a 3×3 conv $\to$ ReLU $\to$ 1×1 conv (already provided).

Implementation Steps:

1. **SimpleCenterPoint**: implement both `__init__` and `forward`. In `__init__`, construct all backbone layers shown in Figure 2 using the provided `ResidualBlock`. Pass `use_batchnorm` through to every block. The stem is a single 3×3 conv (with optional BatchNorm) followed

by ReLU — not a ResidualBlock. In `forward`, apply the blocks in the left-to-right order shown in the figure, then pass the resulting feature map through all five prediction heads (already provided) and return the output dict.

2. **__focal__loss__centerpoint**: the input `pred_hm` is already sigmoid-clamped (done for you in `compute_losses`). Count N_{pos} (pixels where `gt_hm == 1`), then compute:

   - Positive pixels (`gt == 1`): $-(1-p)^2 \log p$
   - Negative pixels (`gt < 1`): $-(1-\text{gt})^4 p^2 \log(1-p)$

   Divide the total sum by $\max(N_{\text{pos}}, 1)$. If $N_{\text{pos}} < 1$, return only the negative term.

3. **__reg__l1__loss**: gather prediction values at the masked center indices using the provided `_transpose_and_gather_feat` helper. Expand the mask, then return the mean absolute error over valid entries, normalised by the mask count.

4. **compute__losses**: apply `_sigmoid_clamped` (provided) to the heatmap predictions, then call the focal loss and the masked L_1 loss for each regression head. Assemble the weighted total using the weights from `train_cfg`. Return a dict with keys `total`, `heatmap`, `reg`, `height`, `dims`, `rot`.

5. **train__step**: zero the optimizer gradients (`set_to_none=True`), run the forward pass `model(bev)`, compute losses via `compute_losses`, call `.backward()` on the total loss, step the optimizer, and return the loss dictionary.

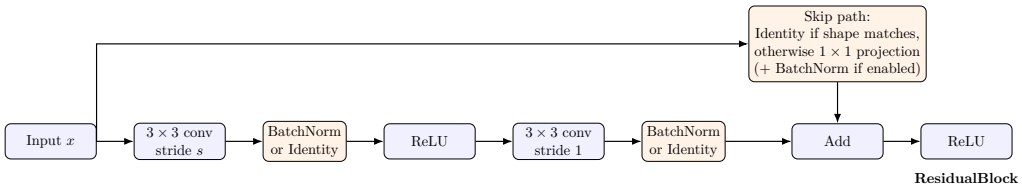

Figure 3: Internal structure of the `ResidualBlock`. With `use_batchnorm=False`, each BatchNorm box becomes an identity map. When stride or channel count changes, the skip path uses a 1×1 projection so that the residual addition is well-defined [1, 2].

Task 3: Inference Decoding [3 pts]

In this task, you will implement the function to decode the network's prediction back to the bounding boxes format.

What you need to implement: In `lidar_det/data/targets.py`:

- `_nms_heatmap` — local-maximum suppression on the sigmoid heatmap.
- `decode_predictions` — full inference decoding from dense head outputs to 3D boxes.

After completion, run:

```
python scripts/03_infer_processed_viz.py \
  --checkpoint outputs/processed_train/task2_minival_debug/best_val.pt \
  --processed-dir data/processed \
  --raw-root data \
  --split minival \
  --max-debug-frames 10 \
  --f1-iou-thresh 0.5 \
  --out-dir outputs/processed_viz
```

Check the output images under `outputs/processed_viz`. The script performs an automatic best-F1 threshold line search and writes the selected threshold to `outputs/processed_viz/best_val/minival/selected_threshold.txt`. Boxes should be aligned with the LiDAR point cloud and the correct perspective in the camera view (Fig. 4)

Implementation Steps:

1. Apply sigmoid and clamp the raw heatmap logits to probabilities in $(10^{-4}, 1 - 10^{-4})$.
2. Suppress non-maximal responses: implement `_nms_heatmap`. Apply `F.max_pool2d` with `stride=1` and padding `(kernel-1)//2` so the output keeps the same spatial size. A pixel is a local maximum if its value equals the pooled value at that location — zero out everything else.
3. Extract the top-k candidate centers with `_topk`; this returns scores, flat indices, class IDs, grid `ys`, and grid `xs`.
4. Gather the regression values at those indices using `_transpose_and_gather_feat`.
5. Refine grid coordinates: add the gathered sub-cell offsets to `xs` and `ys`.
6. Convert to metric coordinates:

$$x = x_{\min} + \texttt{xs} \cdot r, \qquad y = y_{\min} + (H_{\text{out}} - 1 - \texttt{ys}) \cdot r,$$

   where r = `bev_cfg.output_resolution`. Read z directly from the gathered height head. Exponentiate log-dimensions if `use_log_dims=True`. Recover yaw as $\theta = \operatorname{atan2}(\sin\theta, \cos\theta)$.
7. Filter by `score_threshold`, stack into $(N, 7)$ boxes per batch item, and return.

Note: Your `decode_predictions` only needs to perform score-threshold filtering — per-class rotated IoU NMS is handled externally by `class_aware_nms` in the inference script.

Task 4: Full Train/Val [0.5 pt]

After you have verified the data path, target encoding, model, and decoding on `minival`, move to the full processed train/val split.

What you need to implement: Nothing new — this task requires running the training script with well-chosen hyperparameters. Run the command below and monitor the per-epoch validation mAP:

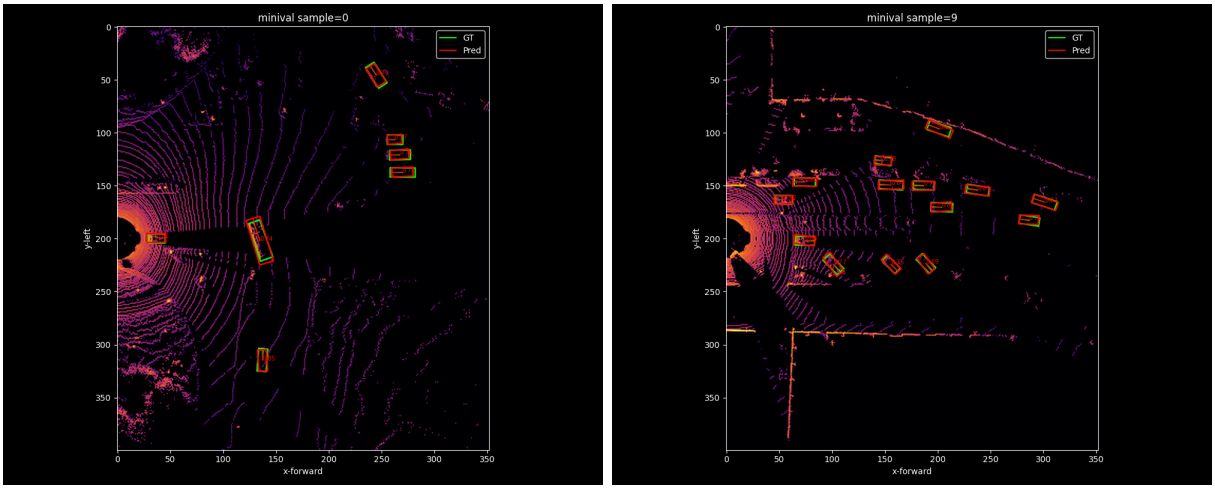

Figure 4: Example of how `decode_predictions` looks like after overfitting on minival split.

```
python scripts/02_train_processed_detector.py \
  --processed-dir data/processed \
  --train-split train \
  --val-split val \
  --test-split test \
  --epochs 10 \
  --batch-size 8 \
  --val-batch-size 8 \
  --eval-iou-threshold 0.5 \
  --run-name task4_full_train
```

You will get the credit if your **test mAP $\geq$ 0.4**, which can be realistically achieved within 5–10 epochs if everything is implemented correctly. The script dumps the checkpoints, a JSON history, and loss-curve plots under `outputs/processed_train/task4_full_train/`.

Hyperparameter guidance: The most important knob is learning rate — lower it if training oscillates or validation mAP stays flat. The script also exposes `weight-decay`, `batch-size`, and `epochs`. For fair comparison across runs, keep `eval-iou-threshold` fixed at 0.5 and do not mix training tuning with post-processing tuning (`score-thresh`, `topk`, `nms-iou`) — treat those as inference-time settings.

While reading your curves, ask not just "is the loss going down?" but "does the detector become more decisive on centers without becoming geometrically worse on box regression?" The per-term losses (`heatmap`, `reg`, `height`, `dims`, `rot`) help answer this.

Bonus Point [0.5pt]: Modify the architecture or training recipe to achieve test mAP of 0.6 or more. Make sure to name your `run-name task4_bonus_point`.

Task 5: Evaluation + PR Curves [0.5 pt]

Evaluate the best validation checkpoint on the held-out test split, save a precision-recall curve, and summarize the metrics at IoU 0.5.

What you need to implement: Nothing new — run the provided evaluation script below and check `outputs/processed_eval/task4_full_train_test/`:

```
python scripts/05_eval_processed.py \
  --checkpoint outputs/processed_train/task4_full_train/best_val.pt \
  --processed-dir data/processed \
  --split test \
  --score-thresh 0.01 \
  --topk 100 \
  --nms-iou 0.1 \
  --iou-threshold 0.5 \
  --out-dir outputs/processed_eval/task4_full_train_test
```

This script writes a text metrics summary (`test_metrics.txt`) and a PR curve image (`test_pr_curve.png`). The PR curve is not decorative: it tells you whether your detector works at one narrow score threshold or maintains a good precision-recall tradeoff across thresholds. Your report should discuss both the scalar AP result and the shape of the PR curve.

Prepared Dataset

The processed dataset is already prepared and split at the drive level. Available drives with id ≤ 56 go to train, ids 57 through 59 go to validation, and all remaining available drives go to test. A deterministic 10-sample `minival` subset is drawn from validation for debugging.

Split	Frames	Artifact Size
Train	2297 (4594 with augmentation)	6.4G
Val	734	2.1G
Test	1234	3.5G
Minival	10	60M

Every processed `.npz` file stores four arrays: `inputs`, `outputs`, `boxes`, and `raw`. The first two are the rasterized tensors discussed earlier. The `boxes` payload stores the non-rasterized box annotations used for evaluation and visualization. The `raw` payload stores a small debug subset with RGB images and point clouds so that Task 3 can project boxes back into the camera view.

Expected Output Structure

```
outputs/
  plots/
    01_kitti_raw/
      *.png
    02_targets/
      *.png
  processed_train/
    task2_minival_debug/
      best_val.pt
      last.pt
      config.json
      history.json
      loss_curves.png
```

```
      val_metrics.txt
    task4_full_train/
      best_val.pt
      last.pt
      config.json
      history.json
      loss_curves.png
      val_metrics.txt
      test_metrics.txt
      val_pr_curve.png
      test_pr_curve.png
    [optional] task4_bonus_points/
      best_val.pt
      last.pt
      config.json
      history.json
      loss_curves.png
      val_metrics.txt
      test_metrics.txt
      val_pr_curve.png
      test_pr_curve.png
  processed_viz/
    best_val/
      minival/
        selected_threshold.txt
        bev/
          *.png
        cam2/
          *.png
  processed_eval/
    task4_full_train_test/
      test_metrics.txt
      test_pr_curve.png
```

Deliverables

Submit the following as part of a zipped file. Do **not** include the `data/` directory or `outputs/tensorboard`.

- Completed `bev.py`, `targets.py`, and `simple_centerpoint.py`.
- The entire `outputs/` folder with all expected files (see above).
- A short report pdf containing the following. There are no strict format requirements, as long as you include all the items.
  - Your final train/val loss curves and per-epoch mAP.
  - Held-out test metrics (AP at IoU 0.5).
  - PR curve at IoU 0.5 and a brief interpretation of its shape.
  - Several qualitative visualization examples (use the best-F1 threshold from the line search).
  - Hyperparameters used for your final run.

Bonus Extensions

Bonus A: Tracking with Kalman Filtering and Data Association [+2 pts]. Extend the detector into a simple online multi-object tracker. A strong starting point is the AB3DMOT recipe [8]: treat each predicted 3D box as a measurement, maintain a Kalman-filter state per active track, predict each track forward one step, and associate detections to tracks with Hungarian matching. Expose your key thresholds in code and discuss at least one failure case.

Bonus B: Multi-Class Detection [+1 pt]. Generalize the pipeline beyond the car-only setting to include pedestrians and cyclists. This extension should propagate through label mapping, target generation, heatmap supervision, class-aware decoding, visualization, and evaluation. Demonstrate per-class metrics or PR curves.

Compute Notes

Use GPU for the full train/val experiment. Use `minival` first for fast debugging. CPU execution is acceptable for sanity checks and small smoke tests, but full training on CPU is not the intended workflow.

References

References

[1] Sergey Ioffe and Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *ICML*, 2015. https://arxiv.org/abs/1502.03167

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *CVPR*, 2016. https://arxiv.org/abs/1512.03385

[3] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *ICLR*, 2015. https://arxiv.org/abs/1412.6980

[4] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *ICLR*, 2019. https://arxiv.org/abs/1711.05101

[5] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. In *ICCV*, 2017. https://arxiv.org/abs/1708.02002

[6] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. Objects as Points. 2019. https://arxiv.org/abs/1904.07850

[7] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. Center-based 3D Object Detection and Tracking. In *CVPR*, 2021. https://arxiv.org/abs/2006.11275

[8] Xinshuo Weng, Jianren Wang, David Held, and Kris Kitani. 3D Multi-Object Tracking: A Baseline and New Evaluation Metrics. In *IROS*, 2020. https://arxiv.org/abs/1907.03961